

Codebase Report

Overview

This is a C++20 desktop multiplayer game prototype with two executables:

- game_server: headless authoritative UDP server.
- game_client: SDL3/OpenGL client with input, rendering, and snapshot display.

The project is built with CMake. SDL3 is fetched through FetchContent; OpenGL is linked only into the client.

Build Targets

game_shared

Defined in CMakeLists.txt.

Contains simulation-independent and client/server-shared game logic:

- src/shared/Simulation.cpp
- public includes from src/

Used by both game_client and game_server.

game_net

Contains UDP networking and packet serialization:

- src/net/UdpTransport.cpp
- src/net/Serialization.cpp

Depends on game_shared.

On Windows, links ws2_32.

game_server

Headless terminal server executable:

- src/server/main.cpp
- src/server/ServerApplication.cpp

Depends on:

- game_shared
- game_net

It intentionally does not depend on SDL3 or OpenGL.

game_client

Graphical game client executable:

- src/client/main.cpp

- src/client/ClientApplication.cpp
- src/client/Input.cpp
- src/client/Camera.cpp
- src/client/GuiLayer.cpp
- src/client/OpenGLRenderer.cpp

Depends on:

- game_shared
- game_net
- SDL3::SDL3
- OpenGL::GL
- imgui

Runtime Flow

1. game_server opens a UDP socket on port 27015.
2. game_client opens an SDL3 window and OpenGL context.
3. Client sends ClientHello to 127.0.0.1:27015.
4. Server records the client endpoint and sends ServerWelcome.
5. Client continuously sends ClientInput packets.
6. Server applies the latest input to authoritative Simulation.
7. Server ticks at fixed 60 Hz.
8. Server sends ServerSnapshot packets.
9. Client updates camera/debug state from the latest snapshot.
10. Client renders the grid through OpenGLRenderer, then renders the crosshair and debug overlay through Dear ImGui.
11. On client shutdown, client sends Disconnect.
12. Server clears the connected client on explicit disconnect or timeout.

Shared Module

src/shared/Math.hpp

Contains minimal math primitives and helpers.

struct Vec2

Two-dimensional float vector.

Members:

- float x
- float y

Used for movement input axes and look delta input.

struct Vec3

Three-dimensional float vector.

Members:

- float x
- float y
- float z

Used for player position and renderer camera calculations.

operator+(Vec3 left, Vec3 right)

Adds two Vec3 values component-wise.

operator*(Vec3 value, float scalar)

Scales a Vec3 by a float.

length(Vec2 value)

Returns Euclidean length of a Vec2.

normalize(Vec2 value)

Returns a normalized Vec2. If length is near zero, returns {} to avoid division by zero.

clamp(float value, float minValue, float maxValue)

Restricts a float to a min/max range. Used to clamp camera pitch.

src/shared/PlayerState.hpp

Defines authoritative player state shared by server and client.

struct PlayerState

Members:

- std::uint32_t playerId
- Vec3 position
- float yawRadians
- float pitchRadians

Default position is {0.0f, 1.8f, 0.0f}, representing an eye-height-like camera/player origin.

src/shared/Protocol.hpp

Defines packet constants and payload structs.

protocolVersion

Current protocol version. Value: 1.

defaultServerPort

Default UDP server port. Value: 27015.

enum class PacketType

Packet identifiers:

- ClientHello
- ServerWelcome
- ClientInput
- ServerSnapshot
- Disconnect

struct ClientInputCommand

Input command sent from client to server.

Members:

- sequence: client-side packet/input sequence number.
- clientTick: monotonically increasing client tick counter.
- movement: Vec2 movement axis from WASD.
- lookDelta: Vec2 camera look delta from mouse or arrow keys.

struct ServerSnapshot

Authoritative snapshot sent from server to client.

Members:

- sequence: server-side snapshot sequence.
- serverTick: authoritative simulation tick number.
- player: authoritative PlayerState.

src/shared/Timestep.hpp

Defines fixed simulation timing.

fixedTickSeconds

Fixed timestep duration. Value: 1.0 / 60.0.

src/shared/Simulation.hpp/.cpp

Authoritative gameplay simulation.

class Simulation

Owns current player state, latest input command, and tick counter.

Members:

- PlayerState player_
- ClientInputCommand latestInput_

- std::uint64_t tickCount_

applyInput(const ClientInputCommand& input)

Stores the latest input command. The simulation currently uses only the most recent command rather than buffering or replaying inputs.

tick(float fixedDeltaSeconds)

Advances authoritative simulation by one fixed step.

Behavior:

- Applies look delta to yaw and pitch.
- Clamps pitch between -1.5 and 1.5 radians.
- Normalizes movement axis.
- Converts movement into world-space forward/right vectors using yaw.
- Moves player at 4.5 meters per second.
- Increments tickCount_.

Constants inside function:

- mouseSensitivity = 0.0025f
- moveSpeedMetersPerSecond = 4.5f
- maxPitchRadians = 1.5f

player() const

Returns current authoritative PlayerState.

tickCount() const

Returns current simulation tick count.

Networking Module

src/net/NetworkTransport.hpp

Defines the abstract transport boundary. This is the seam where future SteamNetworkingSockets support can replace UDP.

struct NetworkEndpoint

Represents a remote network address.

Members:

- std::string host
- std::uint16_t port

Currently assumes IPv4-style host strings.

struct NetworkPacket

Represents a received packet.

Members:

- NetworkEndpoint from
- std::vector<std::uint8_t> bytes

class NetworkTransport

Abstract interface for packet transport.

virtual bool open(std::uint16_t localPort)

Opens a local endpoint. Server uses 27015; client uses 0 for an ephemeral local port.

virtual void close()

Closes the transport.

virtual bool sendTo(const NetworkEndpoint& endpoint, const std::vector<std::uint8_t>& bytes)

Sends raw bytes to a remote endpoint.

virtual std::optional<NetworkPacket> receive()

Receives one packet if available. Returns std::nullopt when no packet is available.

src/net/UdpTransport.hpp/.cpp

UDP implementation of NetworkTransport.

class UdpTransport

Final concrete UDP transport.

Members:

- std::intptr_t socket_

Copying is disabled because it owns an OS socket.

UdpTransport()

Default constructor.

~UdpTransport()

Calls close().

open(std::uint16_t localPort)

Creates a non-blocking UDP socket.

Behavior:

- Closes any existing socket.
- Starts Winsock on Windows.
- Creates IPv4 UDP socket.
- Binds to INADDR_ANY and the requested local port.
- Sets the socket non-blocking.
- Stores socket handle.

Returns false on socket, bind, or non-blocking setup failure.

close()

Closes the socket if open.

sendTo(...)

Sends a byte vector to an IPv4 endpoint.

receive()

Attempts to receive one UDP datagram. Uses a fixed 1400-byte buffer and returns `std::nullopt` when no data is available.

src/net/Serialization.hpp/.cpp

Binary packet serialization/deserialization.

Current format is raw native binary layout for primitive fields. It is simple but not yet portable across endianness or ABI differences.

serializeClientHello(std::uint32_t sequence)

Creates a ClientHello packet containing a sequence number.

serializeClientInput(const ClientInputCommand& input)

Creates a ClientInput packet containing sequence, client tick, movement, and look delta.

serializeDisconnect(std::uint32_t sequence)

Creates a Disconnect packet containing a sequence number.

serializeServerWelcome(std::uint32_t sequence, std::uint32_t playerId)

Creates a ServerWelcome packet containing sequence and assigned player id.

serializeServerSnapshot(const ServerSnapshot& snapshot)

Creates a ServerSnapshot packet containing sequence, server tick, and authoritative player state.

readPacketType(const std::vector<std::uint8_t>& bytes)

Reads only the packet header and returns the packet type if the protocol version matches.

deserializeClientInput(...)

Parses a ClientInputCommand.

deserializeServerSnapshot(...)

Parses a ServerSnapshot.

Server Module

src/server/main.cpp

Server entrypoint. Creates game::ServerApplication and calls run().

src/server/ServerApplication.hpp/.cpp

Headless authoritative server application.

class ServerApplication

Owns UDP transport, authoritative simulation, connected client endpoint, disconnect timeout state, and snapshot sequencing.

Members:

- UdpTransport transport_
- Simulation simulation_
- std::optional<NetworkEndpoint> clientEndpoint_
- std::chrono::steady_clock::time_point lastClientPacketTime_
- std::uint32_t snapshotSequence_

run()

Starts the server and enters the main loop.

Behavior:

- Opens UDP port 27015.
- Logs startup.
- Processes incoming network packets.
- Checks client timeout.
- Runs simulation in fixed 1/60 second steps.
- Sends snapshots after each simulation tick.
- Logs tick and player position every 5 seconds.
- Sleeps 1 ms per loop iteration to avoid busy-spinning.

processNetwork()

Consumes all currently available UDP packets.

Packet behavior:

- ClientHello: records client endpoint, updates last-packet time, sends ServerWelcome, logs connection.
- Disconnect: if from current client, logs disconnect and clears endpoint.
- ClientInput: records sender as client if no client is connected; if sender is current client, updates timeout time and applies deserialized input.

checkClientTimeout(std::chrono::steady_clock::time_point now)

If a client is connected and no packet has arrived for 5 seconds, logs timeout and clears clientEndpoint_.

sendSnapshot()

Sends authoritative snapshot to current client. Does nothing if no client is connected.

isCurrentClient(const NetworkEndpoint& endpoint) const

Returns true if the endpoint matches the currently connected client by host and port.

Client Module

src/client/main.cpp

Client entrypoint. Creates game::ClientApplication and calls run().

src/client/ClientApplication.hpp/cpp

SDL/OpenGL/Dear ImGui client application.

class ClientApplication

Owns the client lifecycle, SDL window, OpenGL context, input, camera, ImGui layer, renderer, UDP transport, and current network/debug state.

Members:

- SDL_Window* window_
- SDL_GLContext glContext_
- Input input_
- Camera camera_
- RenderDebugState debugState_
- GuiLayer guiLayer_
- OpenGLRenderer renderer_
- UdpTransport transport_
- NetworkEndpoint serverEndpoint_
- bool running_
- bool transportOpen_

- bool disconnectSent_
- std::uint32_t inputSequence_
- std::uint64_t clientTick_

~ClientApplication()

Calls shutdown().

run()

Runs the client main loop.

Behavior:

- Initializes SDL, OpenGL, Dear ImGui, renderer viewport, and UDP socket.
- Sends ClientHello.
- Each frame resets input deltas, processes SDL events, sends input, receives packets, renders the 3D grid, renders ImGui, swaps buffers, and delays 1 ms.

initialize()

Sets up client platform and rendering.

Behavior:

- Calls SDL_Init(SDL_INIT_VIDEO).
- Requests OpenGL 3.3 core profile.
- Creates a resizable SDL OpenGL window.
- Creates the OpenGL context.
- Enables vsync.
- Enables SDL relative mouse mode.
- Initializes the Dear ImGui SDL3/OpenGL3 backends.
- Opens UDP socket on an ephemeral port.

shutdown()

Sends disconnect, closes UDP transport, shuts down Dear ImGui, shuts down renderer resources, destroys OpenGL context, destroys SDL window, and calls SDL_Quit().

processEvents()

Polls SDL events, forwards events to Dear ImGui, updates viewport on resize, forwards events to Input, and stops the loop if quit was requested.

processNetwork()

Consumes available UDP packets. Marks connection state on ServerWelcome; applies camera/debug state on ServerSnapshot.

sendInput()

Builds a ClientInputCommand from current input state and sends it to the server.

sendDisconnect()

Sends one Disconnect packet if the transport is open and no disconnect was already sent.

src/client/Input.hpp/cpp

Converts SDL events into protocol-level client input commands.

class Input

Tracks keyboard/mouse state.

Members:

- quitRequested_
- forward_
- backward_
- left_
- right_
- lookUp_
- lookDown_
- lookLeft_
- lookRight_
- lookDelta_

beginFrame()

Clears accumulated mouse look delta for the new frame.

handleEvent(const SDL_Event& event)

Processes SDL quit, mouse motion, and keyboard events.

Controls:

- Escape: quit.
- W: forward.
- S: backward.
- A: strafe left.
- D: strafe right.
- Arrow keys: keyboard camera look fallback.

command(std::uint32_t sequence, std::uint64_t clientTick) const

Builds a ClientInputCommand. Encodes WASD movement, mouse look delta, and arrow-key look delta.

quitRequested() const

Returns whether the user requested quit.

src/client/Camera.hpp/cpp

Stores render camera state derived from authoritative player snapshots.

class Camera

Members:

- PlayerState player_

setFromPlayer(const PlayerState& player)

Copies authoritative player state into camera state.

player() const

Returns current camera/player state.

src/client/GuiLayer.hpp/.cpp

Owns Dear ImGui lifecycle, SDL event forwarding, 2D debug UI, crosshair rendering, and future 2D GUI elements.

struct RenderDebugState

Data displayed in the ImGui overlay.

Members:

- bool connected
- std::uint32_t snapshotSequence
- std::uint64_t serverTick
- PlayerState player

class GuiLayer

Owns ImGui context and backend state.

Members:

- bool initialized_

~GuiLayer()

Calls shutdown().

initialize(SDL_Window* window, SDL_GLContext glContext)

Creates the ImGui context, enables keyboard navigation, applies the dark style, and initializes the SDL3/OpenGL3 ImGui backends.

processEvent(const SDL_Event& event)

Forwards SDL events to ImGui_ImplSDL3_ProcessEvent when initialized.

render(const RenderDebugState& debugState)

Starts a new ImGui frame, draws a foreground crosshair, draws the upper-right network debug overlay, and submits ImGui draw data through the OpenGL3 backend.

shutdown()

Shuts down ImGui OpenGL3 and SDL3 backends, destroys the ImGui context, and marks the layer uninitialized.

src/client/OpenGLRenderer.hpp/.cpp

Minimal OpenGL debug renderer.

class OpenGLRenderer

Owns OpenGL debug rendering resources.

Members:

- int width_
- int height_
- unsigned int program_
- unsigned int vertexArray_
- unsigned int vertexBuffer_

~OpenGLRenderer()

Calls shutdown().

setViewport(int width, int height)

Stores viewport size and calls glViewport.

render(const Camera& camera)

Renders one frame.

Behavior:

- Lazily initializes OpenGL resources.
- Clears the screen.
- Enables depth testing.
- Builds a perspective camera from authoritative player position/yaw/pitch.
- Draws a world-space X/Z floor grid.
- Draws red/blue axis hints at world origin.

shutdown()

Deletes OpenGL buffer, vertex array, and shader program if they exist. Must be called before the SDL OpenGL context is destroyed.

initialize()

Loads required OpenGL function pointers through `SDL_GL_GetProcAddress`, creates the shader program, vertex array object, vertex buffer object, and vertex attribute layout.

Internal Renderer Helpers

`OpenGLRenderer.cpp` has private helper functions and types in an anonymous namespace.

OpenGL Function Loading

The renderer manually loads modern OpenGL functions needed for shaders, buffers, VAOs, uniforms, and program management.

struct Vertex

Single debug vertex with position and color.

struct Mat4

Column-major 4x4 matrix backed by `std::array<float, 16>`.

Vector/Matrix Helpers

Private helpers include dot, cross, normalize, multiply, perspective, and lookAt.

Shader Helpers

`compileShader` compiles GLSL shader source and logs compile errors.

`createProgram` creates the simple colored-vertex shader program.

Geometry Helpers

`addLine` appends two line vertices.

drawVertices(...)

Uploads vertices to the dynamic vertex buffer and issues `glDrawArrays`.

Current Limitations

- Networking supports only one connected client.
- UDP packets use raw native binary serialization, not endian-safe/network-byte-order serialization.
- No packet reliability, ordering, retransmit, or jitter buffering.
- Client does not perform prediction or reconciliation.
- Server uses the latest input only; it does not buffer per-tick input.
- `ServerWelcome` is serialized but not fully deserialized by the client.
- Renderer is debug-only and not yet a real scene/rendering abstraction.
- Dear ImGui is currently used for debug UI only; there are no interactive game menus yet.
- Mouse-look may be unreliable under WSL2/WSLg, so arrow keys provide fallback camera

look.

- Server runs forever and currently relies on process termination for shutdown.